

# Chess Bot v1

## SOMMAIRE

- 01 Par où commencer
- 02 Installation
- 03 Vérification
- 04 Jouer contre le bot
- 05 Préparer les données d'entraînement
- 06 Entraîner le modèle
- 07 Utilisation en ligne de commande
- 08 Architecture
- 09 État d'avancement
- 10 Résultats
- 11 Structure du dépôt
- 12 Références

Bot d'échecs basé sur un Transformer, joué **sans recherche** : le réseau regarde la position et choisit un coup directement, sans explorer l'arbre des variantes. Le niveau est mesuré en Elo face à Stockfish bridé.

Projet de substitution au stage, **SEIBOU Naadjath & LAKRA Rajaa**, ECE.  
Dépôt le 30 août 2026, soutenance semaine du 31 août 2026. Inspiré de *Grandmaster-Level Chess Without Search* (DeepMind, 2024).

---

## Par où commencer

| VOUS ÊTES...                   | LISEZ                                                           |
|--------------------------------|-----------------------------------------------------------------|
| en train de découvrir le sujet | <a href="#">GUIDE-PROJET.md</a> : le sujet expliqué de A à Z    |
| en train d'installer le projet | <a href="#">DEMARRAGE.md</a> : pas à pas, sans prérequis Python |
| pressé                         | la section « Installation » ci-dessous                          |

---

## Installation

```
git clone <url-du-depot>
cd chess-bot-v1

python -m venv .venv
.\.venv\Scripts\Activate.ps1      # Windows PowerShell
# source .venv/bin/activate      # macOS / Linux

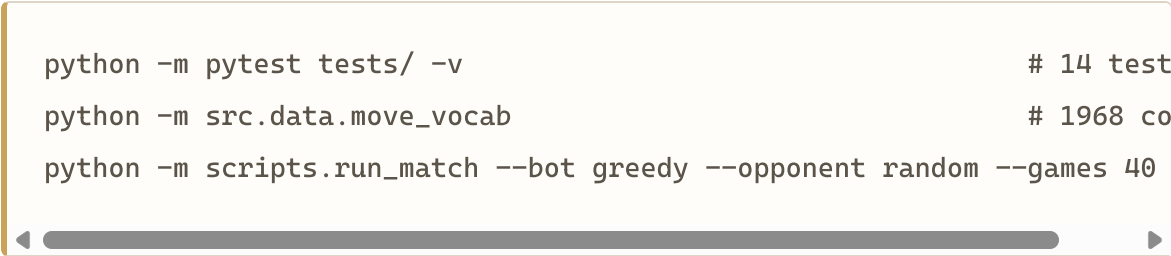
pip install -r requirements.txt
```

Stockfish est un exécutable externe : téléchargez-le sur [stockfishchess.org](https://stockfishchess.org) et placez-le dans `bin/stockfish.exe`. Il n'est pas nécessaire pour les baselines.

---

## Vérification

```
python -m pytest tests/ -v # 14 tests passed
python -m src.data.move_vocab # 1968 combinations
python -m scripts.run_match --bot greedy --opponent random --games 40
```



---

## Jouer contre le bot

```
python -m src.app.server
```

Le navigateur s'ouvre sur `http://127.0.0.1:8000`. Sous Windows, un double-clic sur `Jouer.bat` suffit.

L'interface affiche les coups légaux au survol, l'historique en notation algébrique, et surtout **les coups envisagés par le bot avec leur poids** : cette zone affiche directement les probabilités sorties du Transformer une fois entraîné.

Aucune dépendance web : le serveur repose sur `http.server` de la bibliothèque standard et l'échiquier est dessiné en CSS avec les caractères Unicode des pièces. Le projet démarre donc sans installation supplémentaire, y compris hors ligne.

---

## Préparer les données d'entraînement

```
# Mode démo : teste toute la chaîne en 30 s, sans rien télécharger
python -m scripts.prepare_data --demo-games 60

# Mode réel : archive mensuelle Lichess, lue en flux (pas besoin de dé
pip install zstandard
python -m scripts.prepare_data --input data/raw/lichess_2025-01.pgn.zs
    --max-positions 1000000
```

Filtres appliqués par défaut, chacun justifié dans `src/data/pgn_parser.py` :  
Elo  $\geq 2000$  pour les deux joueurs, cadence  $\geq 3$  min, parties terminées, 8  
premiers demi-coups ignorés, 200 demi-coups maximum par partie.

Le découpage entraînement/validation se fait **par partie et non par position**  
: deux positions d'une même partie se ressemblent trop, les répartir des deux  
côtés constituerait une fuite de données qui gonflerait artificiellement les  
scores.

---

## Entraîner le modèle

```
# Répétition à blanc sur CPU (quelques minutes, valide la chaîne)
python -m scripts.train --epochs 3 --batch-size 64 --d-model 64 --laye

# Entraînement réel, sur GPU (Colab ou Kaggle)
python -m scripts.train --epochs 4 --batch-size 512
```

Le modèle par défaut fait **6,9 M de paramètres** (8 couches, d\_model 256, 8 têtes). Les poids sont sauvegardés à chaque époque dans `checkpoints/` : une session Colab interrompue ne coûte qu'une époque, jamais l'entraînement.

Une fois `checkpoints/best.pt` présent, le Transformer apparaît automatiquement comme adversaire dans l'application, et devient évaluable :

```
python -m scripts.run_match --bot neural --opponent stockfish:1400 --g
```

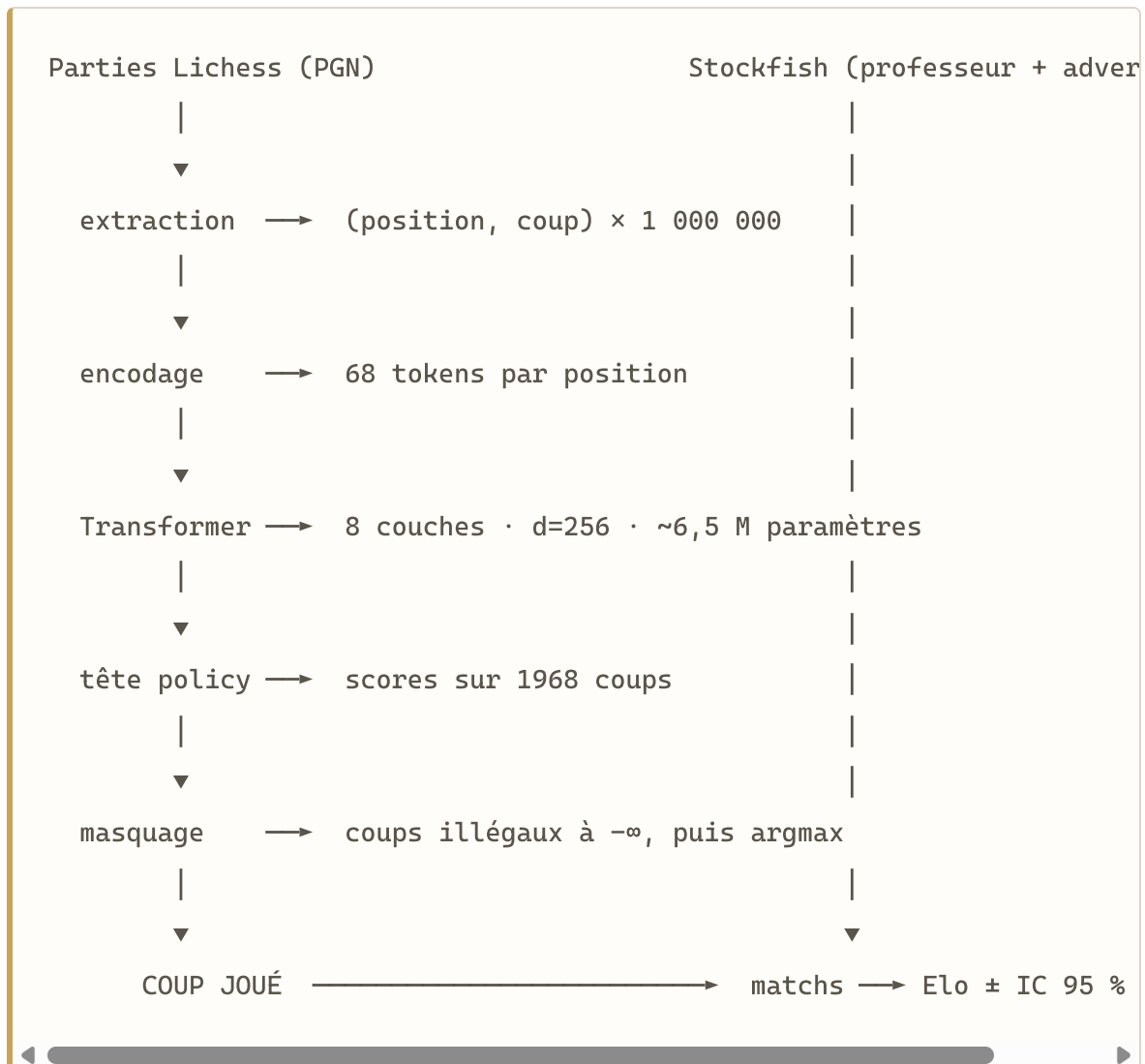
---

## Utilisation en ligne de commande

```
# Tournoi de validation entre les bots de référence
python -m scripts.run_match --tournament --games 40

# Un bot contre Stockfish bridé, parties sauvegardées
python -m scripts.run_match --bot minimax:3 --opponent stockfish:1400
      --games 100 --pgn results/games/minimax_vs_sf1400.pgn
```

# Architecture



## Choix techniques

**Normalisation de couleur.** Le modèle voit toujours la position du point de vue du joueur au trait, comme s'il était les blancs. Quand c'est aux noirs, l'échiquier est retourné et le coup prédit est retourné en retour. Le modèle n'a qu'une seule vue à apprendre → apprentissage environ deux fois plus efficace à quantité de données égale.

**Masquage des coups illégaux.** Les 1968 sorties du réseau couvrent tous les coups géométriquement concevables. À l'inférence, tout ce qui n'est pas dans `board.legal_moves` est mis à  $-\infty$  avant l'argmax : le bot ne peut donc structurellement pas proposer un coup illégal.

**Intervalle de Wilson pour l'Elo.** L'intervalle de Wald classique donne un Elo infini à 100 % de victoires (écart-type nul). Wilson reste borné dans  $[0, 1]$  et fournit une borne basse exploitable même sur un score saturé.

---

---

## État d'avancement

- ✓ Structure du projet, tests, intégration continue locale
- ✓ Vocabulaire des coups (1968) et encodage réversible
- ✓ Baselines : aléatoire, glouton, minimax alpha-bêta
- ✓ Pont Stockfish (UCI), bridage par `UCI_Elo`
- ✓ Moteur de matchs : alternance des couleurs, ouvertures variées, export PGN
- ✓ Estimation Elo avec intervalle de confiance de Wilson
- ✓ Application de jeu (échiquier web, sans dépendance externe)
- ✓ Parser PGN Lichess → dataset (filtres, découpage en tranches, split par partie)
- ✓ Modèle Transformer (6,9 M paramètres) et boucle d'entraînement
- ✓ `NeuralBot` : masquage des coups illégaux, branché dans l'app et l'évaluation

- ✓ Entraînement sur 1 million de positions Lichess (4 époques, GPU T4)
- ✓ Campagne d'évaluation complète contre les baselines et Stockfish bridé
- ✓ Rapport et documentation

---

## Résultats

### Entraînement (4 époques, 1 million de positions Lichess)

| ÉPOQUE | PERTE (VALIDATION) | TOP-1  | TOP-5  |
|--------|--------------------|--------|--------|
| 1      | 3,95               | 12,6 % | 35,4 % |
| 2      | 3,39               | 18,4 % | 45,4 % |
| 3      | 3,14               | 21,9 % | 50,5 % |
| 4      | 3,08               | 22,7 % | 51,7 % |

## Évaluation Elo (60 parties par adversaire, intervalle de Wilson à 95 %)

| ADVERSAIRE           | SCORE  | ELO ESTIMÉ                              |
|----------------------|--------|-----------------------------------------|
| Aléatoire (~250)     | 47,5 % | 233 [146 ; 320]                         |
| Glouton (~600)       | 9,2 %  | 202 [54 ; 349]                          |
| Minimax-2 (~1100)    | 1,7 %  | 392 [88 ; 695]                          |
| Minimax-3 (~1300)    | 1,7 %  | 592 [288 ; 895]                         |
| Stockfish bridé 1320 | 3,3 %  | < 1320 (le bot perd toutes ses parties) |

Niveau constaté : environ 230 à 300 Elo. Le détail complet, avec l'analyse et les parties PGN, figure dans le rapport et dans `results/elo_report.md`.

## Structure du dépôt

|                          |                                                 |
|--------------------------|-------------------------------------------------|
| <code>src/data/</code>   | encodage des positions, vocabulaire des coups   |
| <code>src/engine/</code> | les joueurs (baselines, Stockfish, Transformer) |
| <code>src/eval/</code>   | moteur de matchs et statistiques Elo            |
| <code>scripts/</code>    | programmes exécutables                          |
| <code>tests/</code>      | tests unitaires (pytest)                        |
| <code>tools/</code>      | utilitaires (export HTML de la documentation)   |

---

## Références

1. Ruoss et al., *Grandmaster-Level Chess Without Search*, DeepMind, 2024
2. Vaswani et al., *Attention Is All You Need*, NeurIPS, 2017
3. Silver et al., *Mastering Chess and Shogi by Self-Play (AlphaZero)*, 2017
4. [python-chess](#) · [base Lichess](#) · [Stockfish](#)